# BaccMod, a python package to create background models for IACTs

**Mathieu de Bony,** Marie-Sophie Carrasco, Gabriel Emery, Franca Cassol, and Heide Costantini

CENTRE DE PHYSIQUE DES PARTICULES DE MARSEILLE
CPPM

# BAccMod

- Python package for creating background model for IACTs (requires to be able to read data with gammapy)
- Development started in 2021 with the aim to produce background models for LST-1
- Currently being used in several LST-1 analyses
- Designed to easily integrate in a gammapy workflow

# Capabilities of BAccMod

- Create background model from data
- Multiple available methods
- Zenith binning, azimuth splitting of the models
- Dynamic energy binning

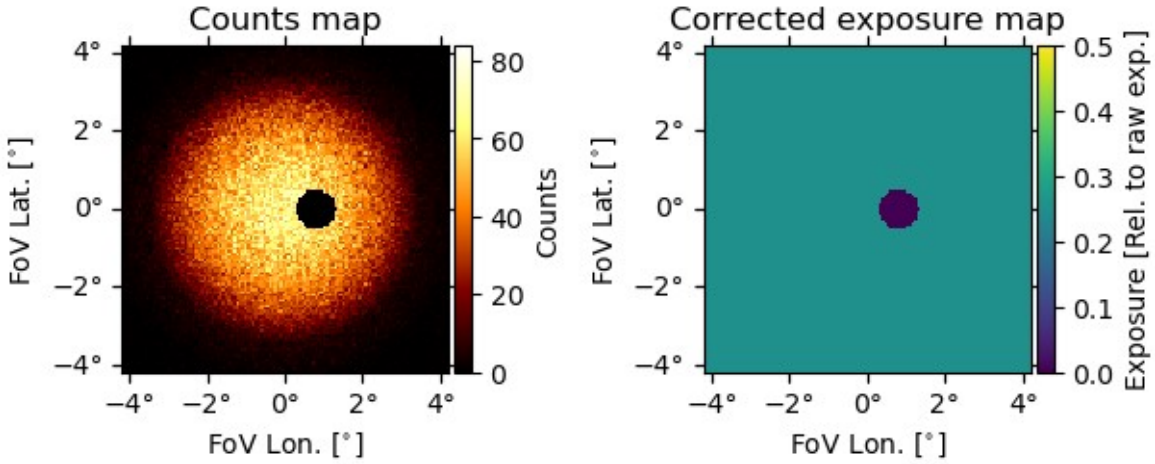
# Available types of model

- 2D counts (energy, offset)
- 3D counts (energy, FoV lon, FoV lat)
- 3D spatial fit (energy, FoV lon, FoV lat)
  (fit of a spatial model in each energy bin)

# Model computation

**2D counts** (per energy bin):

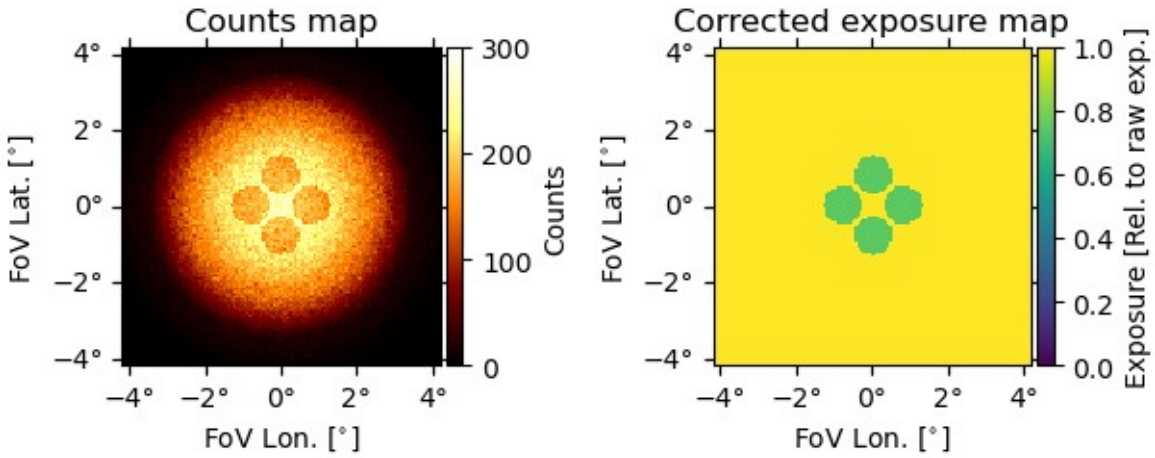
1) **Computes a finely binned map per run**



# Model computation

**2D counts** (per energy bin):

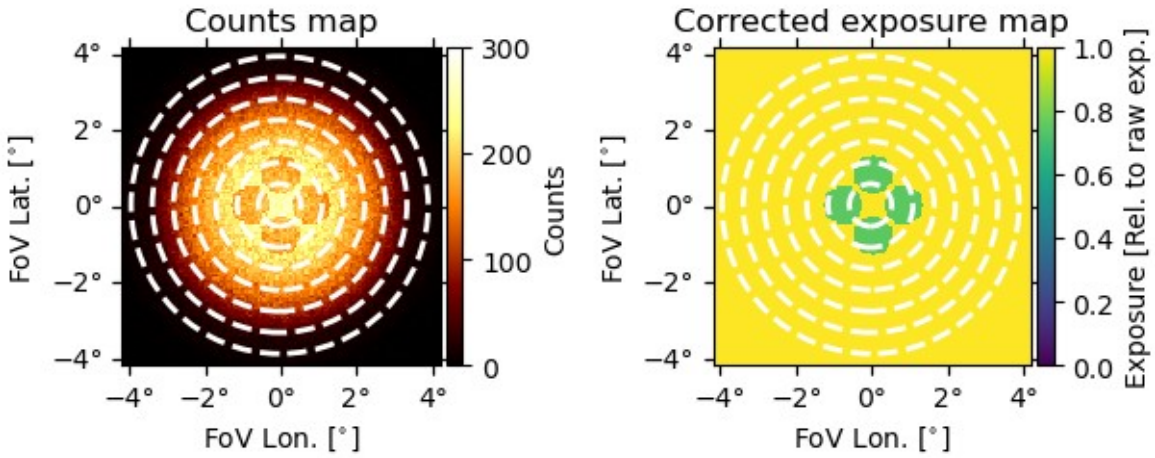
1) Computes a finely binned map per run
2) **Stacks all the runs**



# Model computation

**2D counts** (per energy bin):

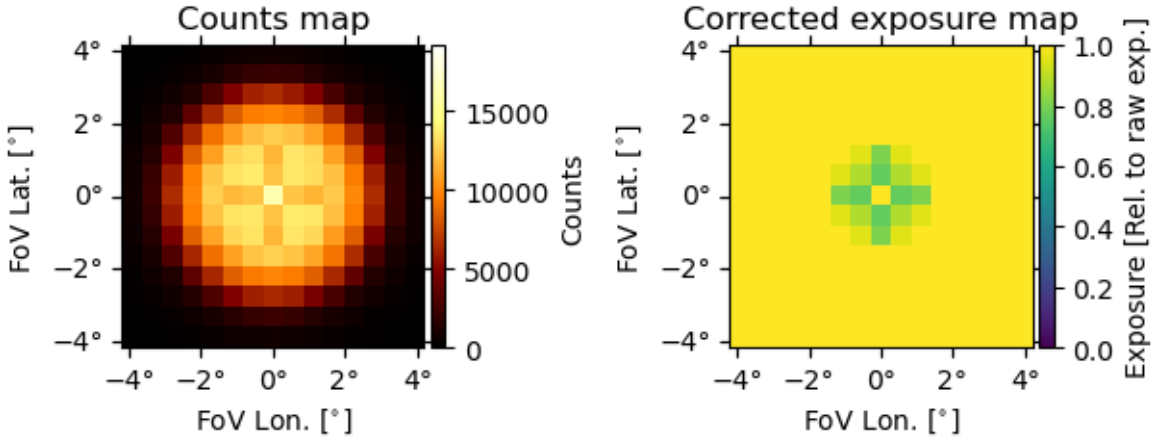
1) Computes a finely binned map per run
2) Stacks all the runs
3) **Sums over offset rings**



# Model computation

**3D counts** (per energy bin):

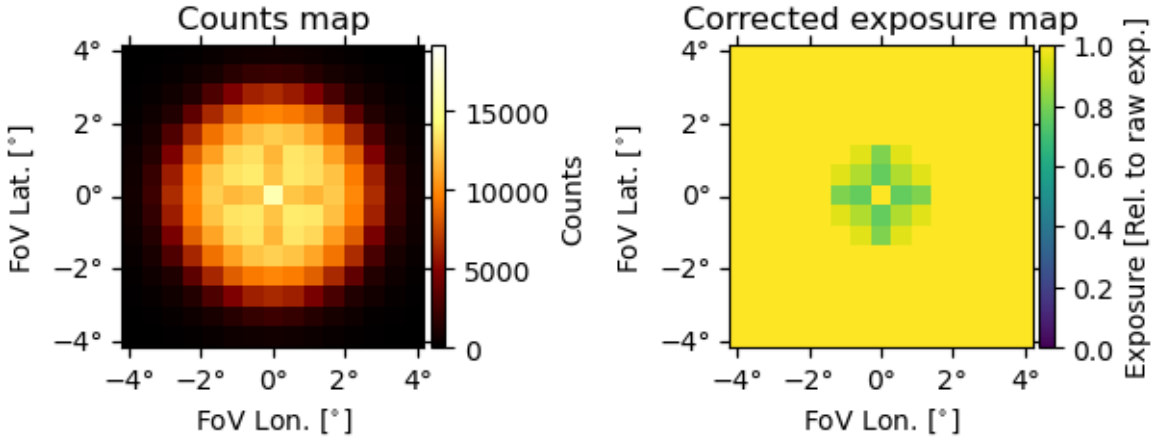
1) Computes a finely binned map per run
2) Stacks all the runs
3) **Down-samples the map**



# Model computation

**3D spatial fit** (per energy bin):

1) Computes a finely binned map per run
2) Stacks all the runs
3) Downs-sample the map
4) **Fits the provided spatia shape**
   (poisson likelihood)



# Dynamic energy binning

As a rule of thumb for IACTs:

- **Low energy :** high statistics, more complex spectral shape
  → Requires small energy bins
- **High energy :** low statistics, power-law like spectral shape
  → Requires large energy bins

# Dynamic energy binning

As a rule of thumb for IACTs:

- **Low energy :** high statistics, more complex spectral shape
  → Requires small energy bins
- **High energy :** low statistics, power-law like spectral shape
  → Requires large energy bins

Solution : **Dynamic energy binning**
-> Aim for a **constant statistics per energy bin** (set by the user)
  -> Start from a defined binning and group bins up to reaching the required statistics
-> **Interpolate on a fixed binning** for easier use

# Zenith binning and interpolation

Performance of IACTs evolves as a function of the zenith
-> Automatic zenith binning

# Zenith binning and interpolation

Performance of IACTs evolves as a function of the zenith
-> Automatic zenith binning

1) **Splits observations** as function of finely binned cos zenith bins
2) **Merge zenith bins** up to reaching a given amount of **exposure time**
3) Compute a **model per zenith bin**
4) **Interpolate** models for each **individual runs**

# Zenith binning and interpolation

Performance of IACTs evolves as a function of the zenith
-> Automatic zenith binning

1) **Splits observations** as function of finely binned cos zenith bins
2) **Merge zenith bins** up to reaching a given amount of **exposure time**
3) Compute a **model per zenith bin**
4) **Interpolate** models for each **individual runs**

It's also possible to split azimuths between east and west orientation

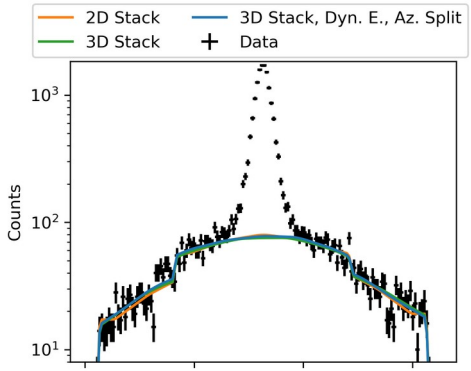
# Upcoming publication

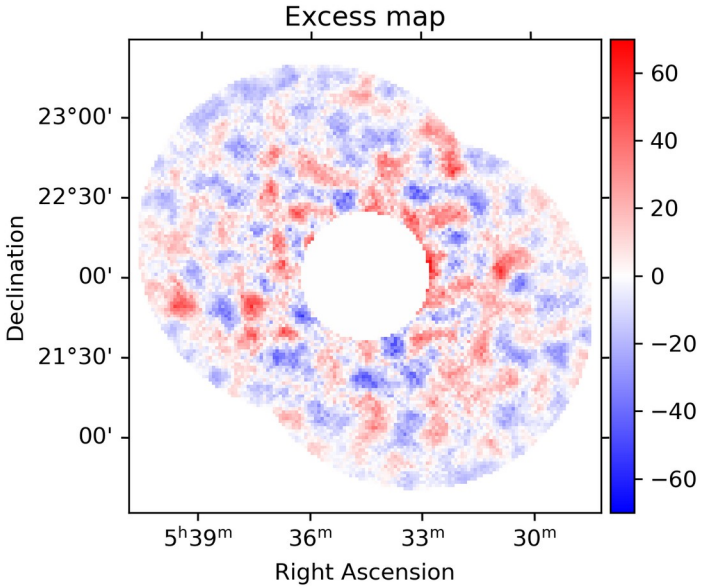
- Detailed explanation of all the functionalities
- Validation on simulated observations
- Validation on LST-1, MAGIC and H.E.S.S. data
- Highlight of the strengths of each method

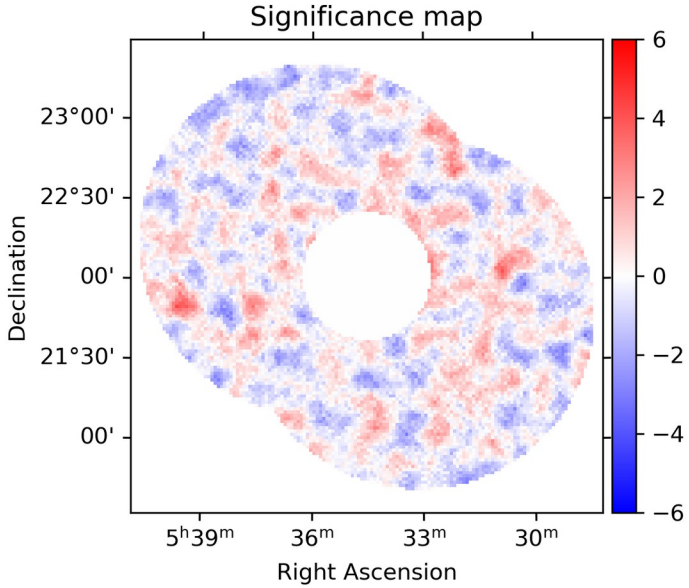
Currently in review within the LST-1 collaboration

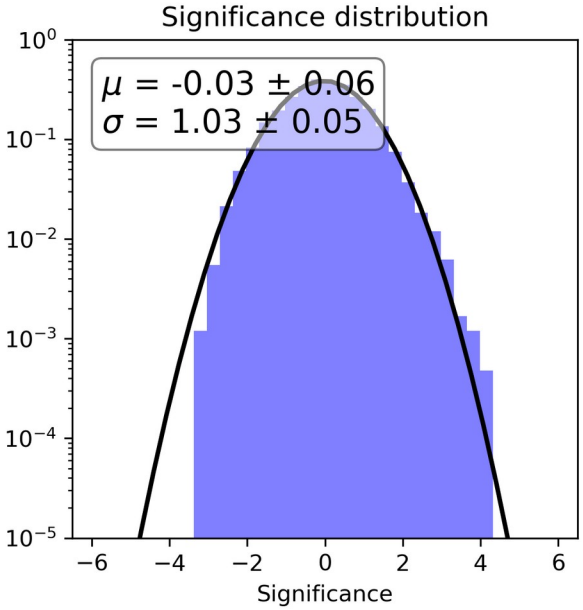
# Example with MAGIC data

- 21h of low zenith Crab data
- FoV Background analysis



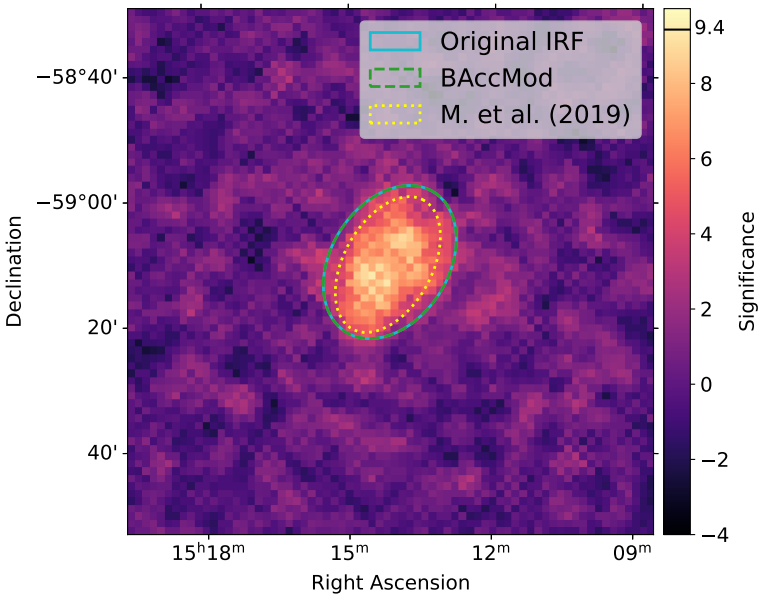



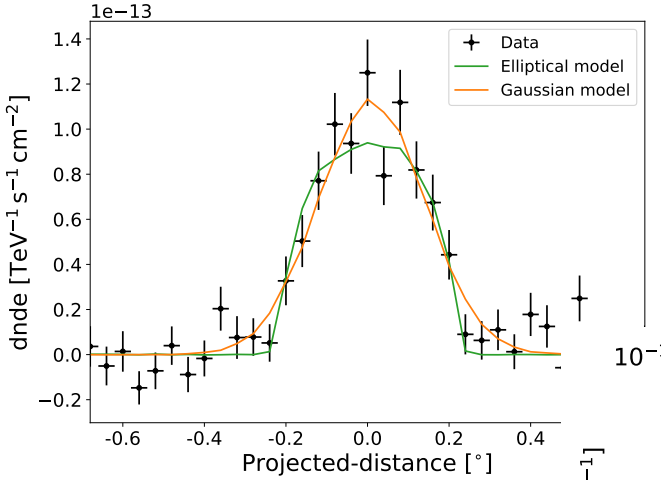



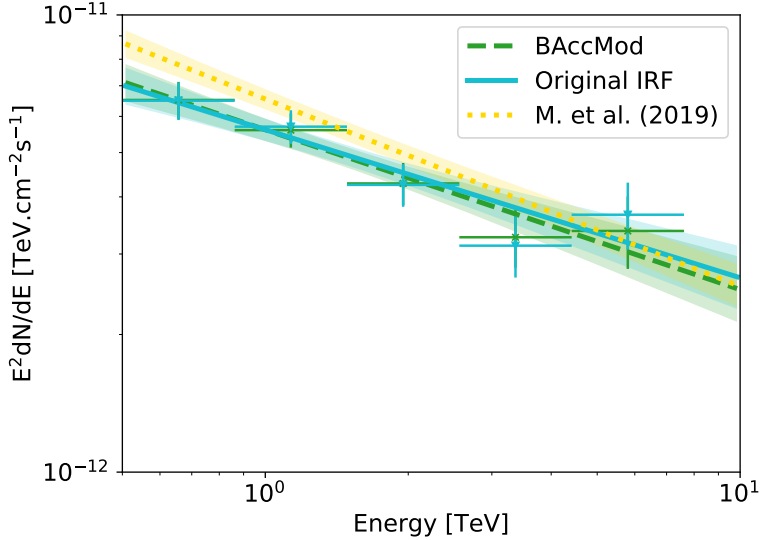



# Example with H.E.S.S data

- 9h MSH 15-52
- 3D analysis



# How to use BAccMod in practice

- Installation with pip : pip install baccmod

# How to use BAccMod in practice

- Load your DL3 with gammapy
- Define offsets and energy bins

```
from gammapy.maps import MapAxis
from gammapy.data import DataStore
from regions import CircleSkyRegion
import astropy.units as u
from astropy.coordinates import SkyCoord
from baccmod import RadialAcceptanceMapCreator

# The observations to use for creating the acceptance model
data_store = DataStore.from_dir("$GAMMAPY_DATA/hess-dl3-dr1")
obs_collection = data_store.get_observations([23523, 23526, 23559, 23592])

# The exclusion regions to apply during acceptance model calculation
exclude_regions = [CircleSkyRegion(center=SkyCoord.from_name('Crab'),
                                   radius=0.2 * u.deg), ]

# Define the binning of the model
e_min, e_max = 0.1 * u.TeV, 10. * u.TeV
size_fov = 2.5 * u.deg
offset_axis_acceptance = MapAxis.from_bounds(0. * u.deg, size_fov, nbin=6, name='offset')
energy_axis_acceptance = MapAxis.from_energy_bounds(e_min, e_max, nbin=6, name='energy')
```

# How to use BAccMod in practice

- Create the model using the gammapy observations
- Return gammapy Background2D or Background3D objects
- API is identical for any of the background methods (a few options are specific)

```
acceptance_model_creator = RadialAcceptanceMapCreator(energy_axis_acceptance,
                                                      offset_axis_acceptance,
                                                      exclude_regions=exclude_regions)
acceptance_model = acceptance_model_creator.create_model(obs_collection)
```

# Upcoming features

- Improvements on the documentation
- Spectral fit per spatial bin of the model
- 3D fit of the model
- Add optional metadata to the output allowing for more details on the internal compute performed by BaccMod (like exact energy binning, counts maps, exposure maps, ...)

# Conclusion

- **BAccMod** is a python package for **IACT background model creation**
- Designed to be integrated into a gammapy workflow
- **Ready to be used** and already being used in several LST-1 analyses
- **Publication coming soon**
- Development is still ongoing to add new features
- Repository : https://github.com/mdebony/BAccMod